



Machine Learning for Petroleum Engineers Using Python

Módulo 7 · Clase 3

Titanic de Punta a Punta — y la Red que Ve



Carlos Enrique Mosquera Trujillo



SLB Ecuador

Dónde Estamos: Dos Actos

Acto 1 — resuelto conmigo:

- **Titanic de punta a punta:** EDA, limpieza, comparación de modelos, probabilidades, explicabilidad y app
- Es **la plantilla de su proyecto** — lo que se entrega el jueves, con su dato

Acto 2 — construido por ustedes:

- ✓ Su primera **red neuronal** — y la pregunta honesta de si le gana al boosting
- ✓ La **no linealidad**, vista con los ojos
- ✓ Una red que **lee dígitos** — y una que **ve fotos** sin que la entrenemos

El Acto 2 va **guiado como la Clase 1**: la carga del dato viene dada, el prompt está listo para pegar, y el código lo construye el agente dirigido por ustedes. **Proyecto:** equipos de hasta 3, entrega hasta el jueves de la próxima semana.

Acto 1 • Titanic de Punta a Punta

El flujo profesional completo, resuelto en vivo

0 – 55 min

La Pregunta

Los **891 pasajeros del Titanic** — el naufragio mejor documentado de la historia, y por eso el dataset con el que medio mundo aprendió ML. Dos preguntas, puramente técnicas:

1 · ¿Qué separó a quienes sobrevivieron de quienes no?

2 · Para un perfil dado, ¿cuál es su *probabilidad*? — no un «sí o no»: un número entre 0 y 1, con sus porqués.

Y una meta de método: este cuaderno es **el estándar del entregable de su proyecto** — EDA completo, comparación de modelos, resultado honesto, explicabilidad, herramienta. Hoy lo ven entero una vez.

El Mapa: Seis Pasos, Un Flujo



Respecto del flujo que ya conocen, hoy se estrenan dos piezas: **COMPARAR** — no un modelo: varios, con búsqueda de recetas — y **EXPLICAR** — el porqué de cada número, que es lo que se defiende en una reunión.

Paso 1 · El Encargo del EDA

Tengo el DataFrame 'pasajeros': los 891 pasajeros del Titanic. Columnas: survived (0/1), pclass (1a/2a/3a), sex, age, sibsp (hermanos/cónyuge a bordo), parch (padres/hijos), fare (tarifa), embarked (puerto), y varias más.

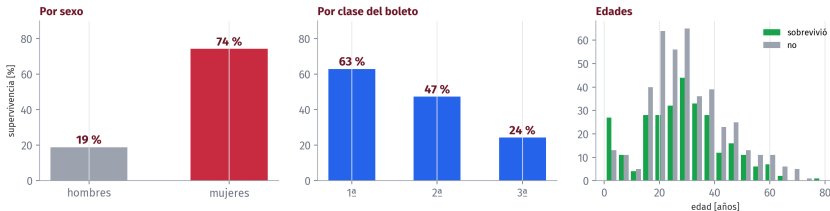
OBJETIVO: un EDA para entender qué separa a quienes sobrevivieron. - estructura y faltantes de cada columna - tasa de supervivencia global, por sexo y por clase - distribución de edades y tarifas - gráficas que se entiendan solas, en español

CRITERIO: al final, dime en una lista qué columnas te parecen sospechosas o redundantes ANTES de modelar.

El criterio del final es el que paga: le pedimos al agente que él mismo levante la mano por las columnas raras. Una de ellas es una trampa mortal — ya la van a ver.

Paso 1 · Lo que Dice el Dato

El EDA en tres hechos: sexo, clase y edad



Tres hechos saltan solos: las mujeres sobrevivieron al **74 %** y los hombres al **19 %** («mujeres y niños primero»), la clase del boleto escalona todo, y entre los niños se sobrevivió más. Faltantes: edad 177, cubierta 688.

La Trampa: el Modelo Perfecto que No Sirve

Entre las columnas sospechosas, el agente señaló a live: dice yes/no... y es **exactamente** survived disfrazado. ¿Qué pasa si se lo damos al modelo?

accuracy = 1.000

Tres líneas de código, cero errores... y cero utilidad: para saber a live hay que saber ya si sobrevivió.

Esto se llama **fuga de información** (*leakage*) y es el error más caro del ML en producción: el modelo brilla en el cuaderno y muere en el mundo real, donde la respuesta no viene incluida en los datos. **La regla que se llevan: si un modelo acierta casi todo, no celebren — desconfíen. Lo perfecto huele a trampa.**

Paso 1 · Las Decisiones de Limpieza, Escritas

1. **Fuera las columnas que repiten o derivan:** `alive` (la fuga), `class` y `embark_town` (duplican en texto), `who`, `adult_male`, `alone` (derivadas de sexo/edad/familia).
2. **Fuera deck:** 688 de 891 faltantes (77 %) — no hay con qué rellenar eso honestamente.
3. **Edad:** 177 faltantes → la mediana. Es una decisión discutible, y por eso se *escribe*.
4. **Puerto:** 2 faltantes → el más común.

Quedan 7 columnas honestas: **clase, sexo, edad, familia a bordo (2), tarifa y puerto**. Con eso se **compite**.

Paso 2 · GridSearch, Desde Cero

Hasta hoy siempre usamos *un* modelo con *una* receta. El flujo profesional prueba **varios modelos con varias recetas** y deja que los datos decidan. La herramienta se llama GridSearchCV, y el nombre se lee por partes:

■ **Grid** — la parrilla: todas las combinaciones de hiperparámetros que uno declara (profundidades, tasas de aprendizaje...).

🔍 **Search** — las prueba todas, una por una.

CV — *cross-validation*: cada receta rinde **cinco exámenes** con cinco particiones distintas del entrenamiento, y se promedia. Así una partición con suerte no corona a la receta equivocada.

Y la regla de siempre, elevada al cuadrado: el examen final del 20 % se aparta ANTES — ninguna búsqueda puede tocarlo. Se usa una sola vez, con el ganador.

Paso 2 · El Encargo de la Comparación

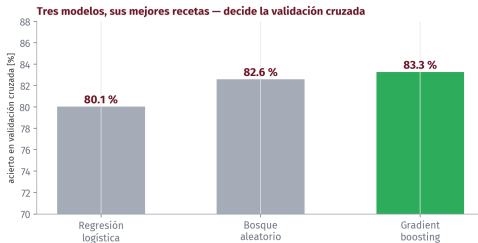
Con el DataFrame 't' limpio (891 filas):

OBJETIVO: comparar tres modelos para predecir survived, con búsqueda de hiperparámetros, y elegir el mejor con honestidad.

RESTRICCIONES: - candidatos: LogisticRegression (con StandardScaler en Pipeline), RandomForestClassifier y HistGradientBoostingClassifier - GridSearchCV con cv=5 y accuracy, random_state = 0 - *reserva ANTES un 20% estratificado, que ningun abs quedapuedetocar - categoricas con get_dummies*

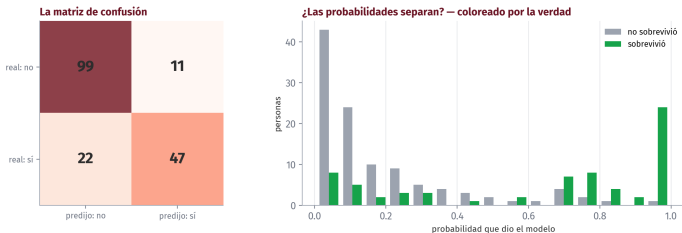
CRITERIO DE ACEPTACIÓN: - una tabla: modelo, mejor receta, accuracy de validación cruzada - el examen final se corre UNA sola vez, con el ganador

Paso 2 · El Resultado de la Búsqueda



Los tres en un pañuelo — 80,1 / 82,6 / 83,3 — y gana el **gradient boosting**: árboles en cadena, cada uno aprende de los errores del anterior (primo del bosque, que vota en paralelo). Y noten lo que NO pasó: **nadie sacó 0.99** — después de la fuga, un número así nos pararía en seco.

Paso 3 · El Examen, en Probabilidades



Examen final: **0.816** (146 de 179). Pero la salida honesta no es «sí/no»: es la **probabilidad** — y la derecha muestra que valen: a las **32** personas a las que dio más de 80 %, sobrevivieron **30** (94 %).

Paso 4 · Explicabilidad: SHAP, Desde Cero

Un 0.91 sin explicación no se puede defender en una reunión. La pregunta que siempre sigue al número es *¿por qué?* — y responderla tiene nombre: **SHAP**.

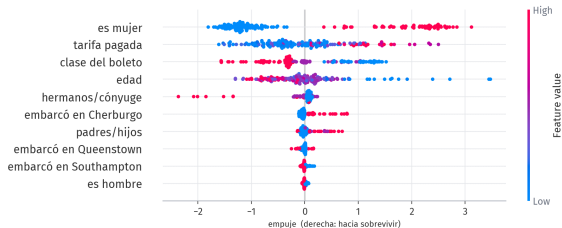
La idea, sin matemática: a cada variable se le asigna la parte *justa* del empujón que dio — cuánto movió la predicción hacia arriba o hacia abajo respecto del pasajero promedio.

Viene de la teoría de juegos (valores de *Shapley*, premio Nobel 2012 de por medio): cómo repartir el premio de un equipo entre sus jugadores según lo que aportó cada uno. Acá el «equipo» son las variables, y el «premio» es la predicción.

Dos niveles, y usamos ambos: **global** (qué manda en el modelo entero) e **individual** (los porqués de UNA predicción — lo que la app le va a mostrar al usuario).

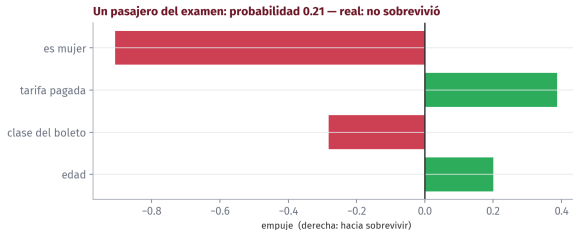
Paso 4 · Qué Mueve la Predicción

Qué mueve la predicción, y hacia dónde — cada punto es un pasajero



Cada punto es un pasajero; derecha = empuja a sobrevivir; el color es el valor (rojo alto). **Ser mujer/hombre parte el gráfico en dos;** la 3ª clase (rojo en «clase») empuja a la izquierda; edad baja empuja a la derecha. **El modelo redescubrió, solo, el protocolo de evacuación de 1912.**

Paso 4 · El Porqué de UNA Predicción



Un pasajero real del examen: probabilidad **0.21**, y sus cuatro factores con signo. Esto es lo que convierte un número en un argumento — y es exactamente lo que la app entrega con cada predicción.

Gradio: Qué Es, y Por Qué Existe



desde 2021, parte de



Hugging Face

Nació en **Stanford, en 2019**, por una frustración que les va a sonar: los investigadores tenían modelos médicos funcionando... y los **médicos** — los que sabían si el modelo servía — no podían probarlos sin pedirle una interfaz a un programador.

La solución fue radical: que **una función de Python se vuelva una página web** con controles, en una línea. Sin saber nada de páginas web.

Hoy es el **estándar de la industria** para poner un modelo en manos de alguien, y ya viene instalado en Colab.

Fíjense para quién se inventó: para que el experto de dominio tocara el modelo sin programar la interfaz. O sea: para ustedes.

Gradio en Dos Escalones — y el Link

La anatomía completa en dos ejemplos de un minuto. **Escalón 1:** toda app son tres piezas — función, entradas, salidas:

```
def saludar(nombre):  
    return f"Hola, {nombre}"
```

```
gr.Interface(fn=saludar, inputs="text", outputs="text").launch()
```

Escalón 2: controles con tipo, y share=True para el **link público** — la dirección que otro abre desde su celular mientras su cuaderno trabaja por detrás:

```
gr.Interface(fn=a_metros_cubicos,  
            inputs=gr.Number(label="Barriles [bbl]", value=1000),  
            outputs=gr.Textbox(label="Metros cúbicos")  
            ).launch(share=True)
```

El link vive mientras el cuaderno corra. Perfecto para una demo o una jornada; no es un sistema permanente.

Pasos 5 y 6 · La App, con Explicación y Contrato

CONTEXTO: tengo 'modelo' (el ganador), 'explicador' (shap.TreeExplainer), 'columnas' (el orden del entrenamiento) y el DataFrame limpio 't'.

OBJETIVO: app de Gradio – perfil del pasajero (clase, sexo, edad, hermanos/cónyuge, padres/hijos, tarifa, puerto) y devuelve: 1. "X de cada 100 personas con este perfil sobrevivieron" 2. una gráfica con los 4 factores que más empujaron ESTA predicción

RESTRICCIONES: - controles en español; rangos de edad y tarifa DESDE el dato 't' - la fila para el modelo se arma como TABLA con las columnas en el orden de 'columnas' (nunca números pelados) - launch(share=True)

CRITERIO: una edad fuera del rango del dato se rechaza con mensaje.

Probado en vivo: edad 500 → rechazada con mensaje. El contrato de entrada, la fila como tabla con nombres, la explicación en la salida — todo lo de la clase pasada, junto.

Acto 2 • La Red que Ve

Guiado: ustedes dirigen, el agente escribe

55 – 115 min

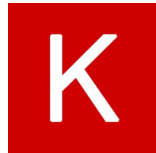
Cómo se Trabaja Este Acto

Cuaderno nuevo: **La Red que Ve**. Guiado como la Clase 1:

- ✓ la **carga del dato viene dada** — esa celda solo se corre
- ✓ cada paso trae su **prompt** listo para pegarle a Gemini
- ✓ la **tarea y el criterio** están escritos — contra eso se revisa lo que devuelva
- ✗ soluciones escritas: **no hay**. Se construye acá.

Las herramientas nuevas del acto: **Keras** (la puerta de entrada a las redes, del equipo de Google) y **Ultralytics YOLO** (detección de objetos).

Ambas gratis; Keras ya está en Colab.



Keras



Ultralytics YOLO

El Puente: ¿Una Red le Ganaría al Boosting?

Si las redes son «lo más avanzado», ¿por qué no usamos una en el Titanic? Respuesta corta: **midámoslo**. El cuaderno les deja el récord a la vista:

EL RÉCORD A BATIR: 0.816 — el boosting, en el examen final

CONTEXTO: tengo X_{ent} , X_{exa} , y_{ent} , y_{exa} (Titaniclimpio, examendel2oesRECORD, deungradientboosting).
OBJETIVO: una red en Keras para la misma tarea, comparada contra el récord EN EL MISMO examen.
RESTRICCIONES: normaliza con StandardScaler (ajustado solo con X_{ent}); doscapasreluysalidasigmoid; earlystopping; semillasfijas.
CRITERIO: los dos números lado a lado, y nada de tocar X_{exa} hasta el final.

Antes de correr: apuesten. ¿Quién gana, y por cuánto?

La Lección Honesta de 2026

En datos tabulares — filas y columnas, como casi todo lo que ustedes manejan — los árboles siguen mandando.

No es una opinión: es el resultado repetido de las competencias y los estudios comparativos de estos años, y lo acaban de medir ustedes mismos. ¿Por qué pasa?

- En una tabla, cada columna significa algo distinto (edad, tarifa, clase) — no hay una *estructura* que la red pueda explotar.
- Los árboles cortan por umbrales («edad < 12») — que es exactamente la forma de las reglas del mundo tabular.
- Con cientos de filas, una red tiene poco de dónde aprender; el boosting exprime más lo poco.

Las redes brillan donde NO hay tabla: imágenes, señales, texto — donde la estructura lo es todo. Vamos allá.

La No Linealidad: la Pregunta

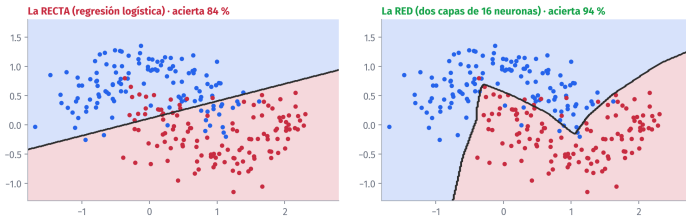
Antes de las imágenes, el concepto que hace distintas a las redes — **visto**, no contado. El cuaderno les genera 250 puntos en dos medias lunas entrelazadas, y la pregunta es: ¿puede una línea recta separarlas?

CONTEXTO: tengo 'puntos' (250 filas, 2 columnas) y 'etiqueta' (0/1) de `make_moons` : *dos medias lunas entrelazadas*.
OBJETIVO: mostrar visualmente qué puede y qué no puede cada modelo. - entrena una `LogisticRegression` y un `MLPClassifier`
(`hidden_layer_sizes` = (16, 16), `max_iter` = 3000, `random_state` = 0) — *dibujalas dos FRONTERAS DE DECISION lado a lado, con los puntos encima, ttulos en español, y el acierto de cada uno en el título*
CRITERIO: que se VEA de un vistazo cuál de los dos puede curvar la frontera.

Tarea: cuando salga, una frase por pregunta — ¿por qué la recta no puede? ¿qué le permite a la red curvarse?

La No Linealidad: la Respuesta, Leída Juntos

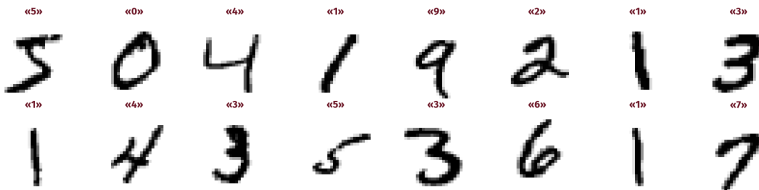
La no linealidad, vista: la frontera que cada modelo PUEDE trazar



La recta acierta **84 %** — su mejor recta posible. La red, **94 %**, porque puede **curvar**. El secreto completo: cada neurona es *suma ponderada + un doblez* (la activación); apilar capas de dobleces permite **componer curvas**. Eso es una red neuronal.

MNIST: Donde No Hay Tabla

MNIST: 70.000 dígitos escritos a mano — acá no hay tabla: hay 28×28 píxeles



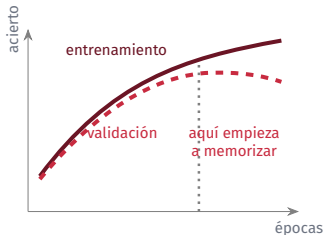
El «hola mundo» de las redes desde hace treinta años: **70.000 dígitos** escritos a mano, cada uno una imagen de 28×28. Aquí no hay columnas con significado: hay **784 píxeles con estructura** — el terreno de la red. Y los píxeles van de 0 a 255: ahí entra la **normalización** (todo a 0–1, dividiendo por 255).

Su Primera Red que Lee

```
CONTEXT0: tengo xent, yent, xexa, yexadekeras.datasets.mnist.Imgenesde28x28conpxeleso  
255; etiquetaso - 9.  
OBJETIVO: mi primera red neuronal en Keras que clasifique los dígitos.  
RESTRICCIONES: - normaliza dividiendo por 255, y aplana cada imagen a 784  
valores - red simple: Dense(128, relu) -> Dense(10, softmax) - compila con adam y  
sparse_categorical_crossentropy - entrena8pocasconvalidation_split = 0.1 - guarda el history en una variable  
CRITERIO DE ACEPTACIÓN: - más de 97- el examen con xexasecorre UNAsolaveza, al final
```

Mientras entrena, miren la barra época por época: están viendo a la red aprender. Ocho épocas, un par de minutos, más del 97 %.

El Overfitting, Visto en Vivo



El esquema. El real lo dibujan ustedes ahora.

Con el 'history' del entrenamiento: grafica accuracy y loss de entrenamiento y de validación por época, lado a lado, en español. Marca la zona donde el entrenamiento sigue mejorando pero la validación ya no.

Tarea: ¿en qué época se separan sus curvas? Esa separación **es** el overfitting: la red memorizando en vez de aprender.

La medicina se llama regularización: parar antes (*early stopping*) o penalizar la complejidad.

La Parte Honesta: ¿En Qué Se Equivoca?

Con la red entrenada y el examen x_{exa} : —la matriz de confusión 10×10 —
muestra 9 dígitos donde la red se equivocó, con lo que ella cree, su confianza, y la etiqueta real

Tarea: miren sus nueve errores y respondan:

- ? ¿Qué pares se confunden — ¿el 4 con el 9? ¿el 3 con el 5?
- ? ¿Los errores son *razonables* — se equivocaría también un humano con esa caligrafía?
- ? ¿La confianza del modelo en sus errores es alta o baja? ¿Qué implicaría eso si esta red leyera cheques?

Mirar los errores de un modelo a la cara es la costumbre que separa a quien lo usa de quien le cree.

El Salto de 2026: No Entrenar — Dirigir

Su red aprendió 10 dígitos con 60.000 ejemplos. Los modelos grandes de visión aprendieron **cientos de clases con millones de imágenes** — y ya están entrenados, gratis, a un pip install de distancia.

YOLO (*You Only Look Once*): una red convolucional que detecta y ubica objetos en una imagen, en tiempo real.

«Convolucional»: en vez de mirar píxel por píxel, aprende *patrones locales* (bordes, texturas, formas) y los compone — la misma idea de apilar dobleces, llevada a imágenes.

OBJETIVO: probar un modelo YOLO preentrenado. - carga el modelo "yolo11n.pt" con ultralytics - detecta objetos en esta imagen de prueba: <https://ultralytics.com/images/bus.jpg> - muestra la imagen con las cajas dibujadas, y debajo un conteo por tipo de objeto, en español

Ahora con SUS Fotos — y la App

Suban una foto del celular (panel  de Colab) y pásenla por el mismo detector.

ANTES DE SUBIR NADA

Nada de instalaciones de la empresa, documentos, ni personas sin permiso. La calle, su escritorio, el parqueadero. El porqué completo, en la Clase 4.

OBJETIVO: una app de Gradio: el usuario sube una imagen y recibe la imagen con las detecciones dibujadas, más el conteo por tipo de objeto en español.

RESTRICCIONES: `gr.Image` de entrada y salida, `launch(share=True)`; si no se detecta nada, decirlo con un mensaje claro.

CRITERIO: que un compañero pueda usarla desde el link, con su foto.

Cierre

El Arco Completo, en Una Tabla

El dato	Quién manda	Lo que hicieron hoy
Tablas (filas × columnas)	Los árboles: bosque, boosting	Titanic: GridSearch, SHAP, app
Imágenes, señales, texto	Las redes neuronales	MNIST: su primera red en Keras
Lo ya entrenado por otros	Ustedes, dirigiendo	YOLO: detectar sin entrenar

Y el mapeo a su mundo, dicho y no ejercitado: detección de EPP en cámaras de planta, lectura de medidores análogos, corrosión en fotos de inspección, conteo de inventario en patio. Todos son «YOLO con ajuste fino» — un curso que ya están en condiciones de tomar. **La herramienta cambia con el dato.**

El método — certificar, examinar, explicar, blindar — no cambia nunca.

Lo Que Aprendimos Hoy

💡 Conceptos:

- ✓ La **fuga de información** — lo perfecto huele a trampa
- ✓ **GridSearch + CV**: que decidan los datos, no el gusto
- ✓ **SHAP**: el porqué global y el de cada predicción
- ✓ La **no linealidad**: neurona = suma + doblez
- ✓ El **overfitting**, visto en sus propias curvas

🔧 Herramientas:

- ➔ GridSearchCV y predict_proba
- ➔ shap — TreeExplainer
- ➔ keras — Dense, relu, softmax, history
- ➔ ultralytics — YOLO preentrenado
- ➔ gradio — por fin, el link

EL NÚMERO DEL DÍA

1.000. El accuracy del modelo con la columna tramposa. El número perfecto era el único imposible.

Los Resultados Negativos de Hoy

Como siempre, en voz alta:

- ✗ **La columna a live daba accuracy 1.000** — y cero utilidad. La fuga de información no avisa: se busca.
- ✗ **La red neuronal no le ganó al boosting** en la tabla del Titanic. En tabular, los árboles mandan — elegir por moda es caro.
- ✗ **Su propia red hizo overfitting** — y lo vieron en sus curvas. Por eso el examen se reserva antes, siempre.
- ✗ **YOLO también falla** — y lo anotaron. Preentrenado no es infalible: el trabajo se movió a dirigir y verificar.

Cuatro fallas medidas en una clase. Eso no es mala suerte: es lo que se ve cuando uno mira. Los que no las ven, las despliegan.

Si Se Llevan Una Sola Cosa

La herramienta se elige midiendo, no por moda — y el número perfecto es el único al que jamás hay que creerle.

Hoy compitieron cuatro modelos y una red. Ganó el que ganó *en el examen*, no el más famoso. Y el único que sacó 100 era una trampa.

Ese par de reflejos — medir antes de elegir, desconfiar de lo perfecto — vale más que cualquier algoritmo de este curso.

Y salieron sabiendo qué es una red neuronal, habiendo entrenado una, y habiendo dirigido a la más famosa del mundo. Nada mal para una tarde.

Su Proyecto: la Plantilla Quedó Lista

El cuaderno del Titanic **es la plantilla de su entrega**. Con su dato, el flujo es idéntico:

1. **EDA** — y la pregunta por las columnas sospechosas
2. **Limpieza** con decisiones escritas
3. **Comparación de modelos** (GridSearch, cv=5)
4. **Examen** reservado antes, corrido una vez
5. **Explicabilidad** — global y por predicción
6. **La app** con link, contrato de entrada y explicación

✓ LA ENTREGA

Hasta el **jueves de la próxima semana**, equipos de hasta 3: el cuaderno con el flujo completo, la app con link, los prompts que usaron, y **un error del agente que ustedes cazaron**. Si su dato es tabular — casi seguro — ya saben qué familia de modelos probar primero, y por qué.

Datos y Referencias

-  **Datos:** pasajeros_titanic en el repositorio del curso (copia exacta del conjunto de práctica de seaborn); **MNIST** vía keras.datasets; imagen de prueba de Ultralytics.
-  **SHAP:** Lundberg & Lee (2017), sobre los valores de Shapley (1953). **Keras:** equipo de Google. **YOLO:** Redmon et al. (2016); implementación moderna de Ultralytics.
-  Las cifras del Acto 1 salen de figuras.py y el cuaderno de la demo las reproduce exactamente. Del Acto 2 **no hay soluciones escritas:** se construye en clase dirigiendo al agente.

Como siempre: corren los scripts y tiene que dar exactamente lo mismo. Si no da, es un error nuestro y queremos saberlo.

¡Gracias!

Carlos Enrique Mosquera Trujillo

✉ cmosquerat@unal.edu.co

Machine Learning for Petroleum Engineers Using Python

Módulo 7 · Clase 3 · SLB Ecuador · UDLA · 2026